Research on Text Classification Based on LSTM-CNN

Hongmin Zhang
College of Information Engineering
Sichuan Agricultural University
Ya'an, Sichuan, China
2111653408@qq.com

Abstract

Text categorization is a crucial task in natural language processing. In recent years, deep learn- ing methodologies, particularly Recurrent Neural Networks (RNN) and Convolutional Neural Net- works (CNN), have made significant advancements in text classification tasks. This paper presents a hybrid model that combines Long Short-Term Memory Network (LSTM) with Convolutional Neural Network (CNN) for improved text classification. The proposed model leverages LSTM to extract long-term dependencies from text sequences, while CNN captures local features. Exper- iments conducted on various publicly accessible datasets demonstrate that this model surpasses other baseline models in performance, affirming its efficacy.

CCS Concepts

• **Computing methodologies** → Artificial intelligence; Natural language processing; Information extraction.

Keywords

Text Categorisation, Deep Learning, LSTM, CNN, Hybrid Models

ACM Reference Format:
Hongmin Zhang. 2024. Research on Text Classification Based on LSTM-CNN. In *2024 5th International Conference on Computer Science and Management Technology (ICCSMT 2024), October 18–20, 2024, Xiamen, China.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3708036.3708084

1 Introduction

With the rapid growth of the Internet, the amount of textual data generated every day has reached unprecedented levels. The proliferation of textual information makes it imperative to devise efficient methods to extract meaningful insights from large and complex datasets. Text categorisation is an important aspect of Natural Language Processing (NLP) that aims to address this challenge by systematically organising text into predefined categories. This categorisation is crucial for a variety of applications such as sentiment analysis, topic classification and spam detection. Each of these tasks requires a nuanced understanding of the patterns and semantics inherent in textual data.

Traditional text classification methods mainly rely on manually designed features, such as bag-of- words models and TF-IDF [1]. However, these methods suffer from feature sparsity and loss of semantic information. In recent years, deep learning techniques, especially Recurrent Neural Networks (RNN) and Convolutional Neural Networks (CNN), have achieved significant results in text classification tasks.

Recurrent Neural Networks (RNNs) have emerged as a robust solution for processing sequential data due to their capacity to preserve contextual information across time steps. Within this realm, the Long Short-Term Memory (LSTM) network stands out as a specialized form of RNN, skillfully engineered to overcome the limitations of standard RNNs, such as the issues of vanishing and exploding gradients. These enhancements empower LSTMs to retain and model long-term dependencies more effectively, which is vital for understanding the semantic structure of extended text sequences and maintaining coherence across long passages.

Concurrently, Convolutional Neural Networks (CNNs), which were originally developed for image processing due to their capability to capture spatial hierarchies in data, have found a new purpose within NLP. In recent years, their application to text classification has garnered significant attention. CNNs excel at extracting local features—such as keywords, n-grams, and contextual phrases—through the application of convolutional filters and pooling operations. This makes them adept at identifying salient details in the text that contribute to its overall meaning and classification.

In this paper, we propose an innovative hybrid model that combines the strengths of LSTM and CNN architectures to improve text classification performance. The proposed method first utilises the LSTM component to efficiently extract long-term dependencies and preserve the semantic flow in the text sequence, thus capturing the overall context. Then, a CNN layer is utilised to perform convolutional operations to accurately locate local features and focus on words and phrases that denote important classification distinctions. The integration of these two mechanisms provides a comprehensive presentation of the input text, taking into account both the overall context and local details.The combined output of the LSTM and CNN is then merged and processed through the fully-connected layer, which maps the learnt features to their respective categories, ultimately producing a robust classification output.

This hybrid model aims to exploit the synergistic potential of LSTMs and CNNs, providing an architecture to address the limitations inherent in using these two approaches alone. The result is a versatile and powerful framework capable of handling the complex requirements of modern text classification, paving the way for further developments in NLP research and applications.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ICCSMT 2024, October 18–20, 2024, Xiamen, China
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0999-9/24/10
https://doi.org/10.1145/3708036.3708084

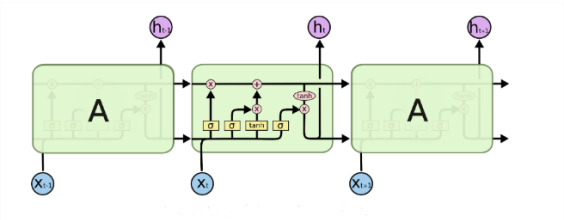

Figure 1: LSTM structure diagram.

2 Related work

2.1 Recurrent Neural Network

The Recurrent Neural Network (RNN) is specifically architected to process sequential data, trans- mitting sequence information via hidden states, thereby enabling the capture of temporal dependencies within sequence data. The Long Short-Term Memory Network (LSTM) (As shown in Figure 1), a distinctive type of RNN, is designed to address the challenges of gradient vanishing and gradient explosion present in traditional RNNs, allowing for more effective capture of long-term dependencies in text sequences.

The core of the LSTM is its memory unit, which contains three gates: an input gate, a forgetting gate and an output gate, as well as a cell state.

- The input gate controls the inflow of new information for the current time step. [2]
- The forgetting gate determines when to forget information in the cellular state.
- The output gate regulates the information released from the current cell state.

The use of these gates allows the LSTM to selectively remember or ignore information at different time steps, thus effectively capturing long-range dependencies. In text categorisation, this allows the model to store important information related to a specific text category and consistently recognise this information across different text inputs.

2.2 Convolutional Neural Network

Convolutional Neural Networks (CNNs) are regarded as foundational algorithms in the domain of deep learning due to their powerful feature extraction and representation capabilities. Typically, CNNs are composed of three primary types of layers: convolutional layers, pooling layers, and fully connected layers. The convolutional layer serves as the backbone of the network, responsible for scanning the input data with multiple filters to identify and capture essential features, such as patterns or specific sequences in the data. This layer extracts these features by performing convolutions, thus creating feature maps that highlight the presence of different elements in the input. [3]

Following the convolutional layer is the pooling layer, which plays a crucial role in reducing the dimensionality of the feature maps. By performing operations such as max pooling or average pooling, this layer effectively downsamples the input, maintaining the most significant information while reducing the overall number of parameters. This downsampling helps to mitigate overfitting by simplifying the network and enhancing its generalization ability.

At the final stage of the CNN architecture, the fully connected layer comes into play. This layer integrates all the high-level features extracted and refined by the previous layers and maps them to the desired output, such as classification labels. By fully connecting each neuron to every input from the previous layer, this stage ensures that the model synthesizes all extracted information to make accurate predictions, completing the process of turning learned representations into actionable insights.

2.3 LSTM-CNN hybrid model

For the past few years, numerous researchers have introduced hybrid models based on LSTM- CNN for text classification. For example, Zhou et al [4] proposed a hybrid model that combines CNN and LSTM for sentiment classification. This model initially employs CNN to discern local features within the text, followed by LSTM to capture the extended dependencies in text sequences. [5]

The LSTM layer processes the entire embedding sequence to help the model understand the overall semantics and context of the text. The CNN layer, on the other hand, performs convolutional operations on the embedding matrix to extract local features such as keyword combinations or phrases, which are often important cues in text classification.

Subsequent to these two layer types, a fully connected layer amalgamates and diminishes the di- mensionality of the acquired features, with the ultimate probability for each category being determined via a softmax layer.

The key advantage of this hybrid architecture is its flexibility and expressiveness. the LSTM layer allows the model to take into account the order and context of the words in the text, while the CNN layer is able to identify more discriminative features in the text. By appropriately adjusting the weights of each type of layer, this architecture is able to find the optimal balance in different types of text clas- sification tasks.

3 Model Description

Neural Networks have been a great achievement in sentence and document modelling, CNN and RNN are the two major approaches, the combination of which yields a network called C-LSTM (As shown in Figure 2) for representing sentences and text classification. C-LSTM uses CNNs to extract high-level phrase representations, which are then inputted to the LSTM to obtain sentence representations. C-LSTM captures the local features of the phrases, the semantic information of the sentences. [6]

3.1 Word Embedding Layer

The word embedding is a class of methods for text representation. It serves the same purpose as one-hot encoding and integer encoding, but has more advantages. There are two word embedding algo- rithms which are word2vec (As shown in Figure 3) and Glove respectively. I am using the first word2vec [7] algorithm here. [8]

3.2 LSTM Layer

The LSTM layer uses the LSTM network to extract long-term dependencies in text sequences. The output of the LSTM is a sequence

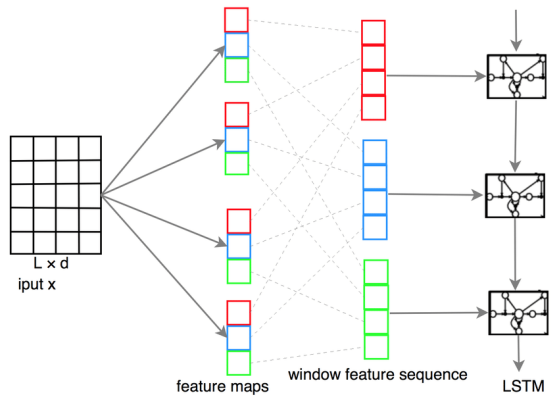

Figure 2: The architecture of C-LSTM.

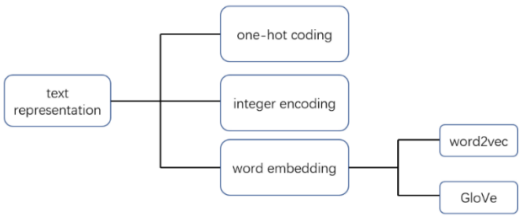

Figure 3: The Place of Word2vec in NLP.

of vectors, each of which represents the semantic information of the corresponding word. [9]

3.3 CNN Layer

The CNN layer employs convolutional and pooling maneuvers to discern local features in the text. The convolution process scrutinizes the text sequence through multiple convolution kernels, extracting features of varying magnitudes. The pooling procedure downscales the convolutional layer's output, preserving the most salient features. [10]

One of the important things about the CNN layer is the convolution. For an input image, it is transformed into a matrix whose elements are the corresponding pixel values. Suppose you have a 5×5 image, and you convolve it with a 3×3 convolution kernel to get a 3×3 feature map. The convolution kernel is also called a filter.

If the input matrix dimensions be denoted as w, the convolution kernel size as k, the step size as s, and the number of zero-padding layers as p, the ensuing feature map's dimensions post-convolution can be determined using the formula:

$$w' = \frac{(w + 2p - K)}{s} + 1 \tag{1}$$

Another key aspect of CNNs is the pooling layer, also known as subsampling or downsampling. This layer is usually located after the convolutional layer in the CNN architecture. The primary function of the pooling layer is to reduce the spatial dimensions of the feature maps generated by the convolutional layer. By compressing these features, the pooling layer effectively minimises the number of parameters in the network, thus generating a more compact and efficient model. This dimensionality reduction not only speeds up computation, but also acts as a regulariser, preventing overfitting by reducing the complexity of the model. By retaining only the most salient features, pooling helps the network to better generalise to new and unseen data, thus improving the robustness and overall performance of the model.

When we talk about pooling operations, we think of the pooling operations that we often use, i.e., Max Pooling and Average Pooling, and in fact there are many kinds of pooling operations. Suppose the input size is i ii, the output size is o oo, the kernel size is k kk, and the stride is s ss, then the following equation is satisfied:

$$o = \frac{i - k}{s} + 1 \tag{2}$$

If there is a pad, the following equation is satisfied:

$$o = \frac{i + 2 \times p - K}{s} + 1 \tag{3}$$

3.4 Fully Connected Layer and Softmax Layer

The outputs produced by the LSTM and CNN layers are combined and fed into the fully connected layer for further processing. This integration allows the model to combine contextual features captured by the LSTM and local features extracted by the CNN to form a comprehensive representation of the input text. The fully-connected layer then uses these combined features and applies Softmax functions to calculate probability distributions for all possible categories. By converting the model's raw output scores into normalised probabilities, the Softmax function enables the model to assign each input to the most likely category, thus making explicit predictions. This step is critical to the classification process to ensure that the output is interpretable and can be used in a variety of applications.

4 Experimental Content

4.1 Data Set

We test our models on various benchmarks. [11] (As shown in Table 1)

- MR:The text dataset used for sentiment analysis contains sentences from film reviews, each la- belled with either a positive or negative sentiment label.
- SST-1:A text dataset for sentiment analysis, created by the Natural Language Processing group at Stanford University. It contains sentences extracted from film reviews, each labelled with one of five sentiment polarity labels (very negative, negative, neutral, positive, very positive)
- SST-2:A simplified version of the SST-1 containing only positive and negative emotions.
- Subj:A textdataset for subjective categorisation that contains sentences from a variety of sources, labelled as 'subjective' or 'objective'.
- TREC:Collected text datasets of various types, including news articles, webpages, emails, social media data, etc., often containing high-quality annotations

Table 1: Data set.

Dataset	N	| v |
MR	10662	18765
SST-1	11855	17836
SST-2	9613	16185
Subj	10000	21323
TREC	5952	9592
CR	3775	5340
MPQA	10606	6246

N: Dataset size. | v | : Vocabulary size.

- CR:Customer reviews of various products (cameras, mp3s, etc.). The task was to predict posi- tive/negative reviews.
- MPQA:Contains multiple datasets, annotations, and tools for studying and evaluating multi- perspective question and answer systems.

4.2 Baseline model selection

In order to comprehensively evaluate the performance of the LSTM-CNN model proposed in this paper on the text classification task, we have selected the following representative baseline models for comparison:

Bag-of-Words Model(BOW):

- Principle: BOW is a simple and widely used text representation method, which treats text as a collection of words, ignoring grammatical and word order information, which is equivalent to repre- senting text as a "bag with words" . We use the word frequency-inverse document frequency method to calculate the word weights, and use logistic regression as a classifier. [12] •Advantages: Simple and intuitive, easy to implement, fast training.
- Disadvantages: Ignores word order and grammatical information, fails to capture semantic rela- tionships between words, and has limited ability to represent text.

Convolutional Neural Networks(CNN):

- CNN model parameter settings: We use a one-dimensional convolutional kernel to process text sequences, set the kernel size to 3, use ReLU as the activation function, and use the Max-Pooling op- eration to extract the most salient features.
- Advantages: Capable of capturing local semantic information in text, such as keywords and phrases.
- Disadvantages: Limited ability to capture long distance dependencies.

Long Short-Term Memory(LSTM):

- Advantages: Capable of capturing long-range semantic information in text, such as sentence- level semantics.
- Disadvantages: More computationally intensive and slower training.

Fairness: to ensure fairness of the comparison, all baseline models use the same experimental setup as the LSTM-CNN models, including:

Table 2: Word2Vec parameters.

parameters	value
sentences	X_all
vector_size	100
window	5
min_count	1
workers	4

Table 3: CNN parameters.

parameters	value
dimension of a word vector	200
kernel_size	3
activation function	Relu
learning rate	0.0002
Drop_out	0.5

- Same word vectors: All models use the same pre-trained word vector model to initialise word embeddings.
- Same dataset partitioning: All models use the same dataset partitioning, including training, val- idation and test sets.
- Same evaluation metrics: All models use the same evaluation metrics to assess performance, such as accuracy, precision, recall, and F1 value. [13]

By comparing with these baseline models, we can more clearly demonstrate the advantages of the LSTM-CNN model on text classification tasks and prove its effectiveness.

4.3 Training Parameter Settings

The Word2Vec parameters are the settings of the word embedding layer and are preprocessing steps that affect the CNN and LSTM layers. Therefore, introducing these parameters first will set the stage for the establishment of CNN and LSTM parameters (As shown in Table 2). To mitigate the risk of overfitting in the CNN (As shown in Table 3) and LSTM network layer (As shown in Table 4), a Dropout rate of 0.5 was employed. This setting deactivates 50% of the neural units at random during each training iteration, preventing the model from becoming overly reliant on specific neurons and promoting more generalized learning. Additionally, the ReLU (Rectified Linear Unit) activation function was utilized to enhance convergence speed by introducing non-linearity and preventing gradient vanishing issues. This combination of techniques—Dropout for regularization and ReLU for efficient training—collectively serves to reduce the risk of overfitting, ensuring that the model maintains robust performance on unseen data by not overfitting to the training set. [14]

4.4 Training Models

Here the LSTM (As shown in Figure 4) and CNN (As shown in Figure 5) models are first trained to see what is the accuracy of each of the two models.

From this we can see that both LSTM and CNN accuracy is around 91%.We then tested the dataset again for classification using

Table 4: LSTM parameters.

parameters	value
dimension of a word vector	200
Hidden Layer Size	50
activation function	Relu
learning rate	0.0002
Out Put Size	2
Drop_out	0.5

```
Epoch [100/100], Batch [200/886], Loss: 0.1486
Epoch [100/100], Batch [300/886], Loss: 0.1140
Epoch [100/100], Batch [400/886], Loss: 0.0973
Epoch [100/100], Batch [500/886], Loss: 0.1246
Epoch [100/100], Batch [600/886], Loss: 0.2067
Epoch [100/100], Batch [700/886], Loss: 0.1309
Epoch [100/100], Batch [800/886], Loss: 0.1202
Test Accuracy: 91.94%
```

Figure 4: LSTM model training result1.

```
Epoch [100/100], Batch [400/886], Loss: 0.0517
Epoch [100/100], Batch [500/886], Loss: 0.1433
Epoch [100/100], Batch [600/886], Loss: 0.0966
Epoch [100/100], Batch [700/886], Loss: 0.2353
Epoch [100/100], Batch [800/886], Loss: 0.1245
CNN model accuracy: 91.51%
```

Figure 5: CNN model training result1.

the LSTM-CNN model.The results obtained are as follows (As shown in Table 5):

The experimental outcomes indicate that the hybrid LSTM-CNN model proposed in this study attains superior classification accuracy across all these datasets, affirming its efficacy.

To better understand the model's performance, accuracy and loss curves for both the training and validation sets were plotted. The results indicated that the LSTM-CNN model exhibited a smooth decline in loss and maintained high accuracy on the validation set, demonstrating its strong generalization capabilities.

5 Robustness testing of the model

In text classification tasks, the robustness of a model is an important measure of its validity and reliability, especially when applied to real-world scenarios. This robustness indicates that the model maintains strong performance even in the face of imperfect data

```
Epoch [100/100], Batch [400/886], Loss: 0.1116
Epoch [100/100], Batch [500/886], Loss: 0.0401
Epoch [100/100], Batch [600/886], Loss: 0.0662
Epoch [100/100], Batch [700/886], Loss: 0.1664
Epoch [100/100], Batch [800/886], Loss: 0.0639
Accuracy after data modification(lstm): 92.33%
```

Figure 6: LSTM model training results.

```
Epoch [100/100], Batch [400/886], Loss: 0.1503
Epoch [100/100], Batch [500/886], Loss: 0.2141
Epoch [100/100], Batch [600/886], Loss: 0.0891
Epoch [100/100], Batch [700/886], Loss: 0.1200
Epoch [100/100], Batch [800/886], Loss: 0.1278
Accuracy after data modification(CNN): 91.70%
```

Figure 7: CNN model training results.

that is common in real-world applications. Textual data often includes noise, inconsistencies, or outliers such as spelling errors, irrelevant information, or unexpected wording, which can challenge the predictive accuracy and stability of the model. These factors may adversely affect classification results, leading to potential classification errors and reduced reliability.

Therefore, evaluating and ensuring the robustness of LSTM-CNN models is crucial to measure their resilience and adaptability when dealing with noisy or atypical input data. Studying the performance of these hybrid models under data perturbation conditions, either through simulated noise injection or real-world irregularities, can provide valuable insights into their strengths and potential limitations. Such analyses can help refine the model architecture and training process to ensure consistent performance and greater applicability in diverse and less controlled environments.

To test the robustness of the model, I modified the dataset by changing some of the positive sentiment statements to negative ones, and then tested the model classification accuracy. The final model test results are as follows (As shown in Figure 6 and Figure 7):

From the results we can see that the accuracy of the model does not change much, which shows that our model is relatively robust.

6 Conclusion

In this article, we present an advanced hybrid LSTM-CNN model tailored specifically for text classification. This model strategically

Table 5: Classification accuracy of different models.

Model	MR	SST-1	SST-2	TREC
Bag-of-words model	88.58%	85.41%	89.26%	88.37%
CNN	89.76%	87.24%	90.52%	90.1%5
LSTM	90.23%	88.17%	91.94%	90.86%
LSTM-CNN	91.54%	89.66%	92.18%	91.37%

combines the strengths of Long Short-Term Memory (LSTM) networks, known for their exceptional ability to capture long-term dependencies in sequential data, with the powerful feature extraction capabilities of Convolutional Neural Networks (CNNs). By leveraging LSTM's aptitude for recognizing contextual and sequential relationships within text, paired with CNN's proficiency in identifying local, phrase-level patterns, the model demonstrates superior performance in categorizing textual data. Experimental findings substantiate that this hybrid architecture not only surpasses traditional models but also excels compared to standalone LSTM or CNN models, thereby affirming its robustness and efficacy.

The C-LSTM framework embodies an integrative approach that unites convolutional layers with LSTM networks, enabling the extraction of granular-level text features through the convolutional process, which are subsequently fed into the LSTM layer to analyze deeper contextual dependencies. This design ensures that both local and sequential semantic nuances are captured effectively, resulting in enhanced text classification outcomes.

Future avenues for research in LSTM-CNN-based models abound and promise exciting advancements. One promising direction lies in the exploration of innovative model architectures, potentially involving ensemble methods that combine various deep learning frameworks such as LSTM, CNN, and Transformer architectures. Incorporating models like BERT or GPT, which are designed with attention mechanisms capable of discerning nuanced relationships across lengthy text sequences, could significantly elevate the model's comprehension and classification prowess. Moreover, refining attention mechanisms by adopting techniques like self-attention or scaled dot-product attention could further enhance the model's ability to focus on salient text features, thereby enriching its analytical depth.

Another pivotal aspect for future research is the expansion into multilingual and cross-domain applications, addressing the growing need for models capable of handling varied linguistic and contextual data. Such work could pave the way for models that perform robustly across different languages and dialects, making them applicable to a broader range of text classification tasks. Through these continuous advancements, the field of hybrid LSTM-CNN text classification can achieve greater precision, adaptability, and applicability in the ever-evolving landscape of natural language processing.

References

[1] H. Zhou. "Research of Text Classification Based on TF-IDF and CNN-LSTM". In: Journal of Physics: Conference Series 2171. 2022. DOI: 10.1088/1742-6596/2171/1/012021.

[2] Yue Li, Xutao Wang, and Pengjian Xu. "Chinese Text Classification Model Based on Deep Learning". In: Future Internet 10. 2018, p. 113. DOI: 10.3390/fi10110113.

[3] S. Kiranyaz *et al.* "1D Convolutional Neural Networks and Applications: A Survey". In: ArXiv abs/1905.03554. 2019. DOI: 10.1016/j.ymssp.2020.107398.

[4] Chunting Zhou *et al.* "AC-LSTM Neural Network for Text Classification". In: ArXiv abs/1511.08630. 2015. URL: https://api.semanticscholar.org/CorpusID:15500867.

[5] HaşimSak,Andrew Senior, and Françoise Beaufays. Long Short-Term Memory Based Recurrent Neural Network Architectures for Large Vocabulary Speech Recognition. 2014. arXiv: 1402. 1128 [cs.NE].

[6] Xiangyang She and Di Zhang. "Text Classification Based on Hybrid CNN-LSTM Hybrid Model". In: 2018 11th International Symposium on Computational Intelligence and Design (ISCID). Vol. 02. 2018, pp. 185–189. DOI: 10.1109/ISCID.2018.10144.

[7] A. Rehman *et al.* "A Hybrid CNN-LSTM Model for Improving Accuracy of Movie Reviews Sentiment Analysis". In: Multimedia Tools and Applications 78. 2019, pp. 26597–26613. DOI: 10.1007/s11042-019-07788-7.

[8] Xin Rong. word2vec Parameter Learning Explained. 2016. arXiv: 1411.2738 [cs.CL].

[9] Ting Huang, Gehui Shen, and Zhi-Hong Deng. Leap-LSTM: Enhancing Long Short-Term Mem- ory for Text Categorization. 2019. arXiv: 1905.11558 [cs.CL].

[10] Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level Convolutional Networks for Text Classification. 2016. arXiv: 1509.01626 [cs.LG].

[11] Yoon Kim. Convolutional Neural Networks for Sentence Classification. 2014. arXiv: 1408.5882 [cs.CL].

[12] Yan YaYa. "A Comparative Study of Bag-of-Words Models and TF-IDF in Text Categorisation". In: Computer Knowledge and Technology 17.28. 2021, pp. 138–140. ISSN: 1009-3044. DOI: 10.14004/j.cnki.ckt.2021.3024.

[13] Beakcheol Jang *et al.* "Bi-LSTM Model to Increase Accuracy in Text Classification: Combin- ing Word2vec CNN and Attention Mechanism". In: Applied Sciences. 2020. DOI: 10 .3390/ app10175841.

[14] Jinbao Teng *et al.* "Multi-Channel Attention Mechanism Text Classification Model Based on CNN and LSTM". In: Computer Engineering and Applications 57.23. 2021, pp. 154–162. ISSN: 1002-8331.